EXPERIMENT-59(HDOC)

C Programming

Program: Anagram Check Using Pointers and Character-Frequency Counting

Step 1: Understand the Problem Statement

- Input: Two strings s and t, each containing only lowercase characters.
- Output: "Anagram" if both strings contain the same characters with the same frequencies, otherwise "Not Anagram".
- The strings are read in main() and passed to `check_anagram(char *s, char *t)` using their addresses; inside the function the strings are accessed only through pointers. The function returns nothing and prints the result itself.
- A frequency array of 26 integers (one per letter a–z) is used: every character of s adds 1 to its counter and every character of t subtracts 1. If all 26 counters end at 0, the character frequencies match.
- The order of characters is irrelevant (counting ignores position), and strings of different lengths are treated as not anagrams.

Rule(s) used: Two strings are anagrams if and only if they have the same length and, for every letter, the count in s equals the count in t. Index of a letter in the frequency table = character − 'a'.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`s = listen` `t = silent`	Anagram	Anagram	**Pass**
2	`s = hello` `t = world`	Not Anagram	Not Anagram	**Pass**
3	`s = anagram` `t = nagaram`	Anagram	Anagram	**Pass**
4	`s = rat` `t = car`	Not Anagram	Not Anagram	**Pass**
5	`s = abc` `t = abcd`	Not Anagram	Not Anagram	**Pass**

Step 3: Algorithm

1. Start the program.
2. Declare two character arrays s and t, read both strings, and call check_anagram(s, t), passing their addresses.
3. In check_anagram(), declare an integer array freq[26] and initialise all elements to 0.
4. Using pointer p, traverse s until '\0' and increment freq[*p - 'a'] for each character.
5. Using pointer q, traverse t until '\0' and decrement freq[*q - 'a'] for each character.
6. Compare the lengths (p − s and q − t). If they differ, print "Not Anagram" and return.
7. Scan freq[0] to freq[25]. If any element is non-zero, print "Not Anagram" and return.
8. If all elements are zero, print "Anagram".
9. Stop the program.

Evaluation against Step 2 test cases: tracing the algorithm for "listen"/"silent" leaves every counter at 0 (l, i, s, t, e, n each +1 then −1), so the output is "Anagram"; for "hello"/"world", the counters for h, e, l (net +1) and w, r, d (net −1) are non-zero, so "Not Anagram"; "anagram"/"nagaram" cancel completely (Anagram); "rat"/"car" leaves t and c non-zero (Not Anagram); "abc"/"abcd" have different lengths (Not Anagram) — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-60: Anagram check using pointers and character-frequency counting.

   The two strings (lowercase only) are read in main() and passed to
   check_anagram() using their addresses. Inside the function, ONLY pointers
   are used to access the strings. A frequency table of 26 letters is built:
   +1 for every character of s, -1 for every character of t. If every count
   ends at 0, both strings have identical characters with identical
   frequencies, so they are anagrams.
*/

#include <stdio.h>

#define SIZE 100

/* Function prototype */
void check_anagram(char *s, char *t);

int main() {
    char s[SIZE], t[SIZE];

    printf("Enter string s: ");
    scanf("%s", s);
    printf("Enter string t: ");
    scanf("%s", t);

    /* pass the addresses of the strings to the function */
    check_anagram(s, t);

    return 0;
}
```

```c
/* Prints "Anagram" if s and t are valid anagrams, else "Not Anagram" */
void check_anagram(char *s, char *t) {
    int freq[26] = {0};
    int i;
    char *p, *q;

    /* count characters of s (+1) using pointer p */
    for (p = s; *p != '\0'; p++)
        freq[*p - 'a']++;

    /* count characters of t (-1) using pointer q */
    for (q = t; *q != '\0'; q++)
        freq[*q - 'a']--;

    /* different lengths can never be anagrams */
    int len_s = p - s;
    int len_t = q - t;
    if (len_s != len_t) {
        printf("Not Anagram\n");
        return;
    }

    /* if every frequency is zero, the strings are anagrams */
    for (i = 0; i < 26; i++) {
        if (freq[i] != 0) {
            printf("Not Anagram\n");
            return;
        }
    }
    printf("Anagram\n");
}
```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

`cc day60.c -o a.out`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot



- ## Conclusion

In this lab session, pointer-based string manipulation and the character-frequency-counting approach were successfully applied to determine whether two strings are valid anagrams:

1. Pointer Passing & Length Validation: The strings s and t were passed to the function check_anagram() using their base memory addresses via pointers. Initial length checks using strlen() immediately filtered out strings of mismatched lengths.

2. Frequency Mapping: A 26-element integer frequency array (freq[26]) mapped lowercase character occurrences in linear time (\mathcal{O}(N)), incrementing counts for characters encountered in s and decrementing counts for characters in t using pointer offset notation.

3. Verification: Final validation verified that all frequency indices balanced out to zero, ensuring accurate anagram classification regardless of character order.

The program compiled successfully using gcc in a Linux environment, and all test case executions matched the expected output table predictions.